# Αναζήτηση Εικόνων με Διανυσματικές Αναπαραστάσεις (BoVW)

Τελική Τεχνική Αναφορά (Project Report)

Μιχαήλ Ανδρουτσόπουλος (ΑΜ: sdi2200005)
Κωνσταντίνος Τσιότσιας (ΑΜ: sdi2300251)
Τμήμα Πληροφορικής και Τηλεπικοινωνιών
Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών (ΕΚΠΑ)

## Περίληψη

Στην παρούσα εργασία υλοποιήσαμε και αξιολογήσαμε ένα σύστημα αναζήτησης εικόνων με τη μέθοδο Bag-of-Visual-Words (BoVW). Χρησιμοποιήσαμε χαρακτηριστικά SIFT και δημιουργήσαμε ένα οπτικό λεξιλόγιο. Στη συνέχεια, δοκιμάσαμε και συγκρίναμε πέντε διαφορετικούς αλγορίθμους αναζήτησης: την εξαντλητική αναζήτηση (Exact Search) και τέσσερις προσεγγιστικές μεθόδους (LSH, Hypercube, IVF-Flat, IVF-PQ). Στόχος μας ήταν να δούμε πώς συμπεριφέρεται η κάθε μέθοδος όταν η βάση δεδομένων μεγαλώνει (από 1.000 έως 25.000 άσχετες εικόνες) και να μελετήσουμε το πόσο πέφτει η ακρίβεια σε σχέση με τον χρόνο που κερδίζουμε. Τέλος, είδαμε πώς επηρεάζουν τα αποτελέσματα οι αλλαγές στον φωτισμό σε σχέση με τις αλλαγές στην οπτική γωνία.

## 1 Εισαγωγή

Όταν έχουμε μια τεράστια βάση με εικόνες και θέλουμε να βρούμε ποιες μοιάζουν με μια εικόνα που δίνουμε (query), χρειαζόμαστε έξυπνους τρόπους αναπαράστασης και γρήγορους αλγορίθμους. Η μέθοδος Bag-of-Visual-Words είναι μια κλασική λύση σε αυτό. Μετατρέπει κάθε εικόνα σε ένα διάνυσμα που μας δείχνει πόσες φορές εμφανίζεται η κάθε «οπτική λέξη». Με αυτόν τον τρόπο, μπορούμε να μετρήσουμε την απόσταση μεταξύ δύο εικόνων (πόσο μοιάζουν) και να τρέξουμε αλγορίθμους αναζήτησης. Στην εργασία αυτή, γράψαμε τον κώδικα για όλα αυτά τα βήματα και δοκιμάσαμε το σύστημά μας στο dataset HPatches, προσθέτοντας εικόνες από το MIRFlickr-25K για να κάνουμε την αναζήτηση πιο δύσκολη (προσθήκη θορύβου).

## 2 Εξαγωγή Χαρακτηριστικών & BoVW

Το πρώτο βήμα είναι να εξάγουμε τα χαρακτηριστικά (descriptors) από τις εικόνες, χρησιμοποιώντας τον αλγόριθμο SIFT. Για να μη γεμίσει η μνήμη και καθυστερήσει το πρόγραμμα από περιοχές με υπερβολική λεπτομέρεια, κρατήσαμε το πολύ $S = 500$ SIFT descriptors από κάθε εικόνα.

Στη συνέχεια, πήραμε τα descriptors από το training set και τρέξαμε τον αλγόριθμο K-Means. Ο K-Means βρήκε 256 κέντρα (εφόσον επιλέξαμε $K = 256$). Αυτά τα 256 κέντρα αποτελούν το «λεξιλόγιό» μας ($c_1, \ldots, c_K \in \mathbb{R}^{128}$).

Στο τέλος, κάθε εικόνα μετατρέπεται σε ένα ιστόγραμμα (ένα διάνυσμα $x_I$ με 256 θέσεις). Κάθε θέση δείχνει το ποσοστό των SIFT της εικόνας που ανήκουν στην αντίστοιχη λέξη του λεξιλογίου, διαιρώντας με το συνολικό πλήθος των χαρακτηριστικών για να γίνει κανονικοποίηση:

$$n_I = \sum_{j=1}^{K} h_j, \quad x_I = \left( \frac{h_1}{n_I}, \ldots, \frac{h_K}{n_I} \right) \qquad (1)$$

## 3 Παρουσίαση Μεθόδων Αναζήτησης

Για να βρούμε τις 10 πιο κοντινές εικόνες σε κάθε query, φτιάξαμε 5 διαφορετικούς αλγορίθμους:

- **Exact Search:** Παίρνει το query και υπολογίζει την Ευκλείδεια απόσταση ($L_2$) με *όλες* τις εικόνες της βάσης. Είναι η πιο σίγουρη μέθοδος γιατί βρίσκει πάντα τα σωστά αποτελέσματα, αλλά είναι αρκετά αργή όταν η βάση είναι μεγάλη.

- **LSH (Locality Sensitive Hashing):** Χρησιμοποιεί τυχαίες ευθείες προβολής για να φτιάξει πίνακες κατακερματισμού (hash tables). Ομαδοποιεί τα διανύσματα σε buckets. Έτσι, όταν ψάχνουμε, κοιτάμε μόνο τις εικόνες που έπεσαν στον ίδιο κουβά με το query.
- **Hypercube:** Αντιστοιχίζει τα διανύσματα στις κορυφές ενός φανταστικού υπερκύβου. Αν δε βρει αρκετές εικόνες στην κορυφή που έπεσε το query, αρχίζει να ψάχνει στις διπλανές κορυφές με βάση την απόσταση Hamming.
- **IVF-Flat (Inverted File):** Τρέχει ξανά έναν K-Means πάνω στα διανύσματα της βάσης για να τα χωρίσει σε ομάδες (συστάδες Voronoi). Όταν έρχεται ένα query, βρίσκει σε ποια ομάδα ανήκει και ψάχνει μόνο μέσα σε αυτήν (και σε μερικές γειτονικές, ανάλογα με την παράμετρο $nprobe$).
- **IVF-PQ (Product Quantization):** Μοιάζει με το IVF-Flat, αλλά επιπλέον «κόβει» τα διανύσματα σε μικρότερα κομμάτια και τα συμπιέζει. Αυτό γλιτώνει πάρα πολύ χώρο στη μνήμη και κάνει τους υπολογισμούς απόστασης πολύ πιο γρήγορους, αλλά χάνει λίγο σε ακρίβεια λόγω της συμπίεσης.

# 4 Πειραματική Μελέτη (Test Set)

Γράψαμε ένα Bash script για να αυτοματοποιήσουμε τα πειράματά μας στο Test Set. Αφού κάναμε δοκιμές, καταλήξαμε ότι οι καλύτερες παράμετροι για να έχουμε μια καλή ισορροπία χρόνου και ακρίβειας είναι: `-vocab 256`, `-L 15`, `-probes 100`, `-nprobe 10` και `-M 16`. Στον Πίνακα 1 φαίνονται συγκεντρωμένα τα αποτελέσματα για 1.000, 5.000, 10.000 και 25.000 εικόνες-θορύβου.

## 4.1 Σχέση Ποιότητας και Χρόνου

Βλέποντας τα νούμερα του Πίνακα 1, καταλαβαίνουμε πολλά για το πώς δουλεύει η κάθε μέθοδος στην πράξη:

- **Exact Search:** Όπως περιμέναμε, έχει το καλύτερο Recall (0.808 για D=1000). Όμως, όσο προσθέτουμε εικόνες στη βάση, ο χρόνος ανεβαίνει γραμμικά. Στις 25.000 εικόνες αργεί αισθητά (φτάνει σχεδόν τα 44 ms ανά query), επειδή ελέγχει κάθε εικόνα μία προς μία.
- **IVF-Flat:** Είναι με διαφορά η πιο πρακτική μέθοδος. Κρατάει την ακρίβεια σχεδόν στα ίδια επίπεδα με την Exact (βρίσκει το 73% των σωστών εικόνων στο D=25000), αλλά επειδή κοιτάει μόνο συγκεκριμένες ομάδες, ο χρόνος μένει χαμηλά στα 11.6 ms.
- **LSH και Hypercube:** Αυτές οι δύο μέθοδοι βασίζονται αρκετά στην τύχη των προβολών. Ο Υπερκύβος είναι πολύ γρήγορος (περίπου 1 ms), αλλά δυστυχώς η ακρίβειά του πέφτει άσχημα (στο 0.365) όταν η βάση μεγαλώνει. Αυτό συμβαίνει επειδή τα περισσότερα διανύσματα πέφτουν σε άδειες κορυφές. Το LSH τα πάει αρκετά καλύτερα σε ακρίβεια (0.713), αλλά χρειάζεται περισσότερο χρόνο.
- **IVF-PQ:** Εδώ βλέπουμε ξεκάθαρα το κόστος της συμπίεσης. Επειδή τα διανύσματα δεν είναι ακριβή αλλά προσεγγιστικά, το Recall πέφτει στο 0.66. Αν όμως είχαμε μια τεράστια βάση με εκατομμύρια εικόνες, ο χρόνος του θα έμενε σταθερός, κάνοντάς το ίσως τη μόνη ρεαλιστική επιλογή.

## 4.2 Illumination vs Viewpoint Queries

Ένα πολύ ενδιαφέρον κομμάτι ήταν να δούμε πώς συμπεριφέρονται τα ερωτήματα όταν αλλάζει ο φωτισμός σε σχέση με το όταν αλλάζει η γωνία λήψης της φωτογραφίας (viewpoint). Παίρνοντας τα αναλυτικά αποτελέσματα της Exact Search για $D = 5000$, παρατηρήσαμε τα εξής:

- Στα queries που αλλάζει ο **φωτισμός** (π.χ. το `i_crownday` ή το `i_fruits`), η μέθοδος Bag-of-Visual-Words δουλεύει τέλεια. Βρίσκει και τις 5 σχετικές εικόνες στις πρώτες 5 θέσεις (Recall@10 = 1.0). Αυτό εξηγείται από το γεγονός ότι οι SIFT descriptors είναι φτιαγμένοι έτσι ώστε να μην επηρεάζονται πολύ αν η εικόνα σκοτεινιάσει.
- Αντίθετα, στα queries που αλλάζει η **οπτική γωνία** (όπως το `v_charing` ή το `v_village`), το σύστημα μπερδεύεται αρκετά. Επειδή η κάμερα έχει κουνηθεί, τα

Πίνακας 1: Αποτελέσματα Αναζήτησης (Recall@10 και Μέσος Χρόνος σε ms) για διαφορετικά μεγέθη Βάσης (D)

| **Μέθοδος** | **D = 1000** | **D = 5000** | **D = 10000** | **D = 25000** |
|---|---|---|---|---|
| Exact Search | 0.808 / 1.94 ms | 0.782 / 8.86 ms | 0.773 / 17.55 ms | 0.739 / 43.98 ms |
| IVF-Flat | 0.808 / 1.60 ms | 0.782 / 5.29 ms | 0.773 / 7.11 ms | 0.730 / 11.64 ms |
| LSH | 0.747 / 1.65 ms | 0.747 / 7.52 ms | 0.747 / 13.76 ms | 0.713 / 33.43 ms |
| Hypercube | 0.721 / 0.89 ms | 0.495 / 1.20 ms | 0.434 / 1.44 ms | 0.365 / 1.10 ms |
| IVF-PQ | 0.800 / 11.81 ms | 0.695 / 13.96 ms | 0.713 / 12.83 ms | 0.660 / 13.88 ms |

σχήματα στην εικόνα παραμορφώνονται. Σε αυτά τα queries, το σύστημα έβρισκε συνήθως μόνο 1 ή 2 σωστές εικόνες, ρίχνοντας πολύ τον συνολικό μέσο όρο. Ο θόρυβος από τις 5.000 άσχετες εικόνες έκανε τα πράγματα ακόμα πιο δύσκολα.

## 5 Συμπεράσματα

Ολοκληρώνοντας την εργασία, διαπιστώσαμε στην πράξη ότι όταν έχουμε μια μικρή βάση εικόνων, η απλή εξαντλητική αναζήτηση αρκεί. Όμως, για βάσεις με δεκάδες χιλιάδες εικόνες, είναι απαραίτητο να χρησιμοποιούμε ευρετήρια. Το IVF-Flat ήταν μακράν η πιο ισορροπημένη επιλογή, αφού έδινε σχεδόν τα ίδια αποτελέσματα με την Exact Search σε ένα μικρό κλάσμα του χρόνου. Τέλος, καταλάβαμε ότι το μέγεθος του λεξιλογίου είναι πολύ σημαντικό, γιατί αν βάλουμε πολύ λίγες λέξεις (π.χ. K=100), οι εικόνες δεν ξεχωρίζουν καλά μεταξύ τους και η ακρίβεια πέφτει.